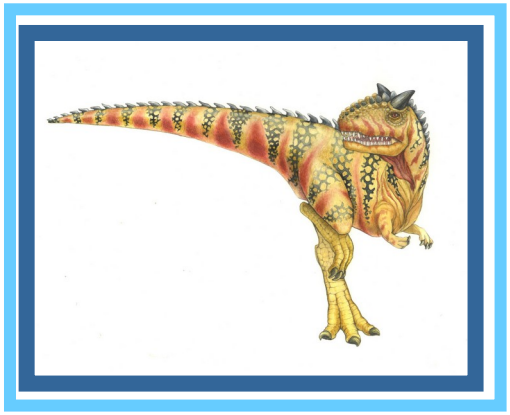
Chapter 6: Process Synchronization



Silberschatz, Galvin and Gagne ©2009

Chapter 6: Process Synchronization

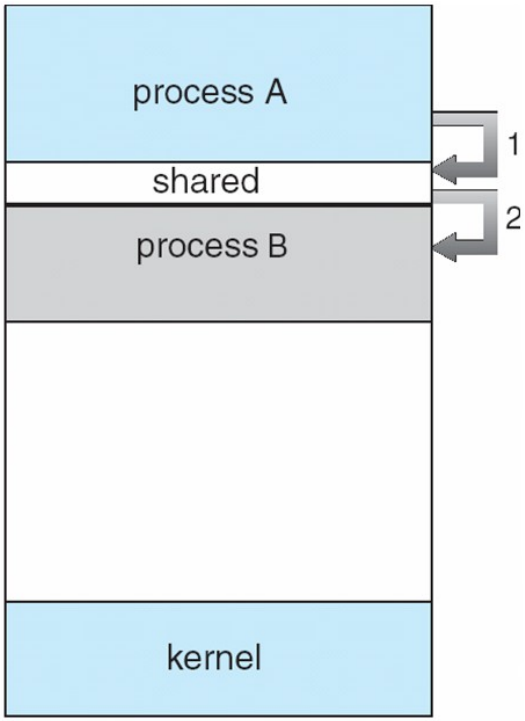
- Background
- The Critical-Section Problem （临界区问题）
- Peterson's Solution （ Peterson 解决方案）
- Synchronization Hardware （同步硬件）
- Semaphores （信号量）
- Classic Problems of Synchronization （同步经典问题）
- Monitors （管程）
- Synchronization Examples
- Atomic Transactions

Background

- Concurrent access （并发访问） to shared data may result in data inconsistency （数据不一致）
- Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes （协作进程顺序执行）

Background

- Example： Suppose that we wanted to provide a solution to the consumer-producer problem that fills all the buffers. We can do so by having an integer count that keeps track of the number of full buffers. Initially, count is set to 0. It is incremented by the producer after it produces a new buffer and is decremented by the consumer after it consumes a buffer.



Producer

Producer :

```
while (true) {
        /* produce an item and put in nextProduced*/
        while (count == BUFFER_SIZE) ; // do nothing
        buffer [in] = nextProduced;
        in = (in + 1) % BUFFER_SIZE;
        count++;
}
```

Consumer

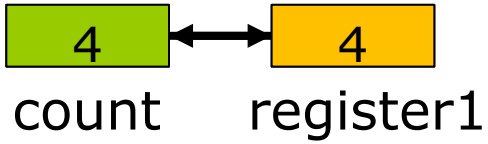
Consumer :

```
while (true) {
        while (count == 0)
            ; // do nothing
        nextConsumed= buffer[out];
        out = (out + 1) % BUFFER_SIZE;
        count--;

        /* consume the item in nextConsumed

}
```

Race Condition (竞争条件)

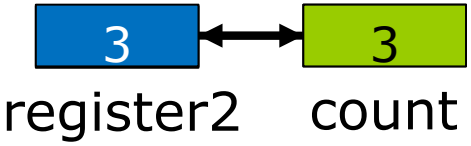
- count++ could be implemented as

register1 = count
register1 = register1 + 1
count = register1



- count-- could be implemented as

register2 = count
register2 = register2 - 1
count = register2



Race Condition (竞争条件)

Consider this execution interleaving with "count = 3" initially:

P1: producer execute register1 = count {register1 = 3}

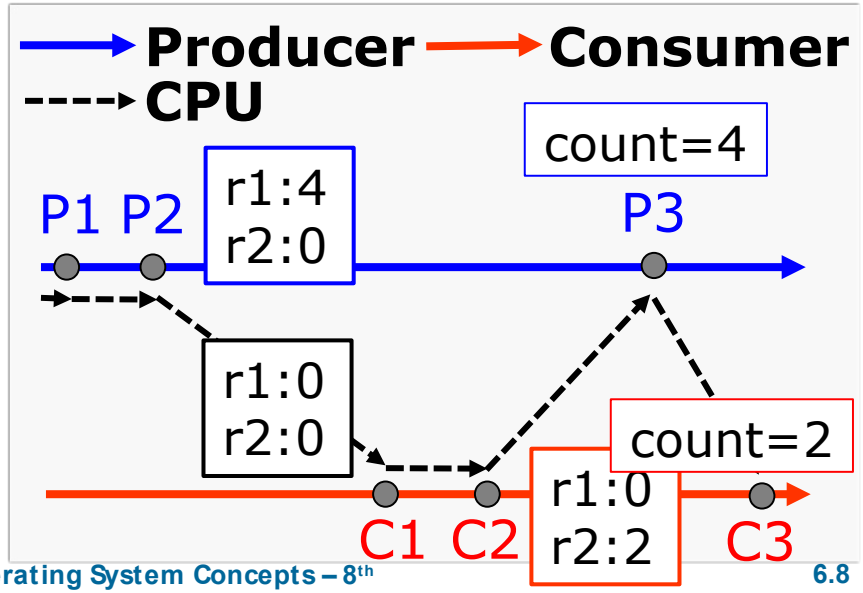
P2: producer execute register1 = register1 + 1 {register1 = 4}

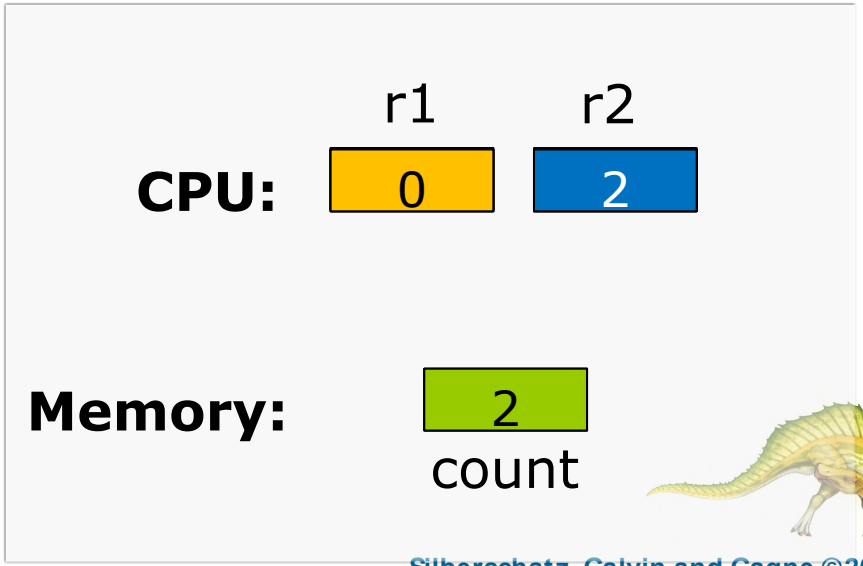
C1: consumer execute register2 = count {register2 = 3}

C2: consumer execute register2 = register2 - 1 {register2 = 2}

P3: producer execute count = register1 {count = 4}

C3: consumer execute count = register2 {count = 2}





Race Condition (竞争条件)

- count++ could be implemented as

  register1 = count
  register1 = register1 + 1
  count = register1

- count-- could be implemented as

  register2 = count
  register2 = register2 - 1
  count = register2

- Consider this execution interleaving with “count = 3” initially:

  P1: producer execute register1 = count {register1 = 3}
  P2: producer execute register1 = register1 + 1 {register1 = 4}
  C1: consumer execute register2 = count {register2 = 3}
  C2: consumer execute register2 = register2 - 1 {register2 = 2}
  P3: producer execute count = register1 {count = 4 }
  C3: consumer execute count = register2 {count = 2}

Race Condition (竞争条件)

- What is a race condition?
  - Several processes access and manipulate the same data concurrently （原因：多个进程同时访问相同数据）
  - The result of the execution depends on particular access order. （后果：执行结果与执行顺序有关）
- To avoid race condition, we need to ensure that only one process at a time can be manipulating the same data.----mutual exclusion(互斥访问)

Two threads, one counter

Popular web server

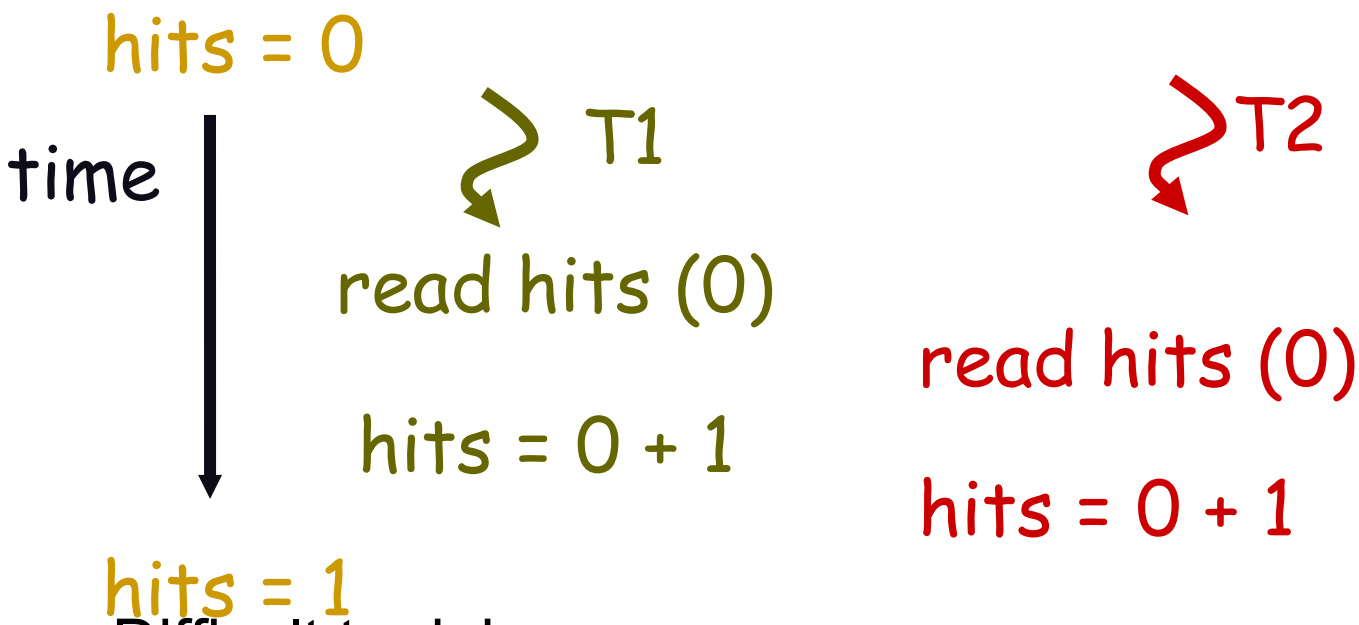
- Uses multiple threads to speed things up.
- Simple shared state error:
  - each thread increments a shared counter to track number of hits

```
...
hits = hits + 1;
...
```

- What happens when two threads execute concurrently?

Shared counters

- Possible result: lost update!



- ⇒ Difficult to debug
- Called a “race condition”

Objectives

- To introduce the critical-section(临界区) problem, whose solutions can be used to ensure the consistency of shared data
- To present both software and hardware solutions of the critical-section problem （软件和硬件解决方法）
- To introduce the concept of an atomic transaction and describe mechanisms to ensure atomicity （原子事务）

Critical-Section (临界区)

Critical resource （临界资源）：

- The resource that must be accessed mutual excluded (互斥的).

Critical section （临界区）：

A segment of code in which the process is accessing critical resource

Consider a set of concurrent processes{ $P_0, P_1, \ldots, P_{n-1}$}.

Each process has a segment of code, called a critical section, in which the process may be changing common variables, updating a table, writing a file, and so on.

Critical-Section

- To ensure the shared data to be modified mutual excluded, no more than one process can execute in its critical section at the same time. （不能有一个以上的进程同时在它的临界区执行）
- Each process must request permission to enter its critical section----entry section (进入区)
- And exit with the exit section (退出区).

```
entry section
  critical section
exit section
  remainder section
```

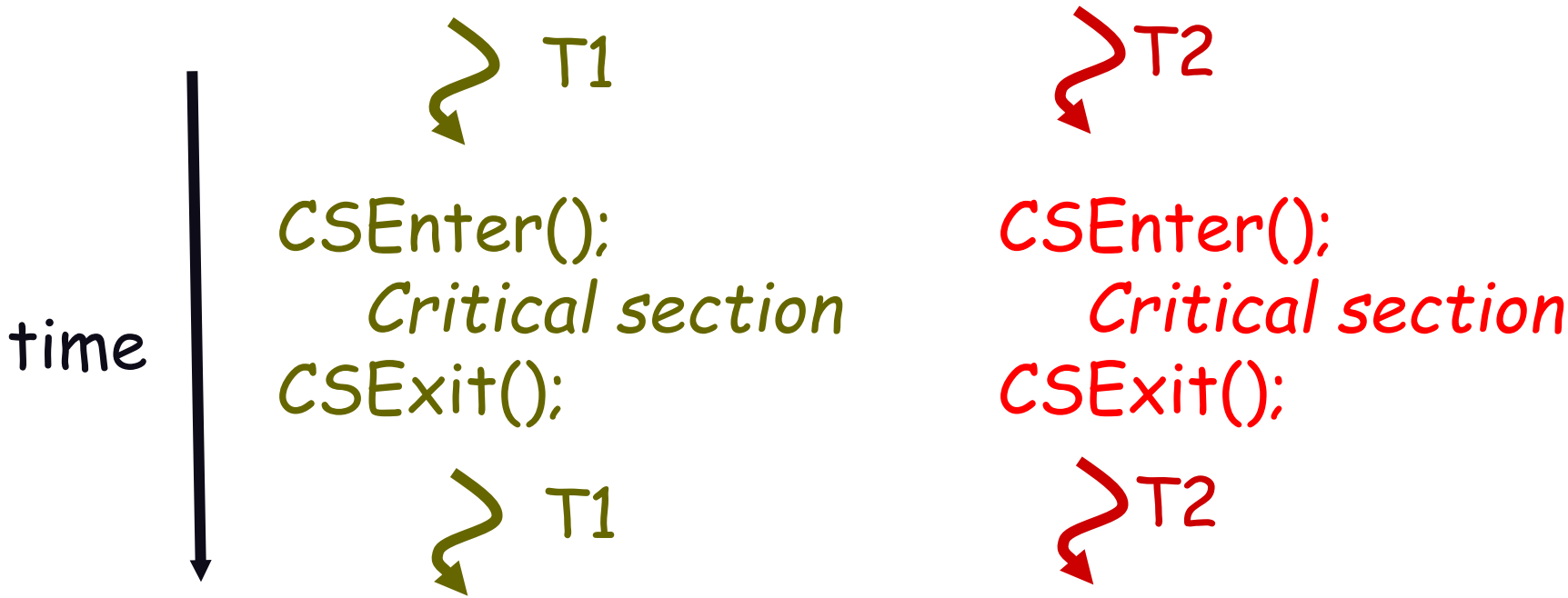
Solution to Critical-Section Problem

1. Mutual Exclusion 互斥 (safety) - No more than one process can be in a critical section at any time.
2. Progress 有空让进 (liveness) - If no process is executing in its critical section and there exist some processes wishing to enter their critical section, then the selection of the processes that will enter the critical section next cannot be postponed indefinitely
3. Bounded Waiting 有限等待 - A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted
   - Assume that each process executes at a nonzero speed
   - No assumption concerning relative speed of the N processes

- Ideally we would like fairness （公平） as well
  - If two threads are both trying to enter a critical section, they have equal chances of success
  - … in practice, fairness is rarely guaranteed

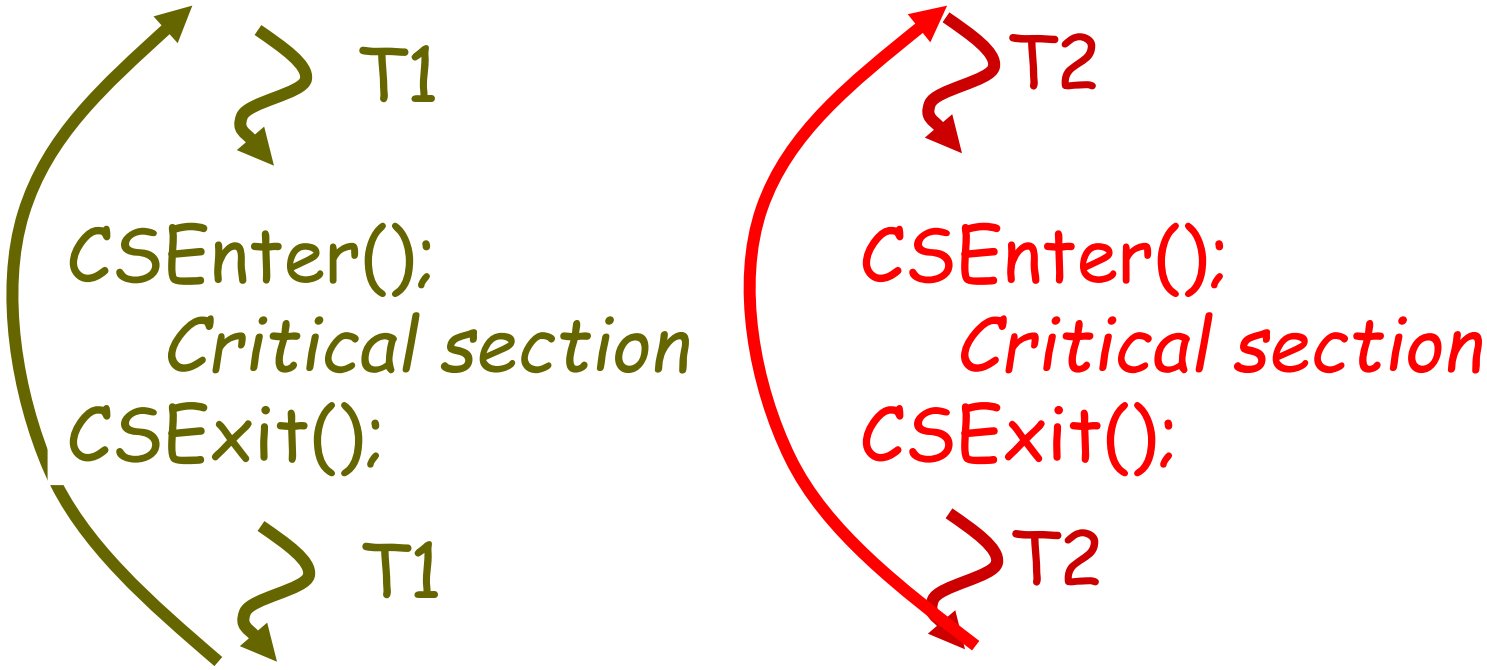
Critical Section Goals

- Threads do some stuff but eventually *<u>might</u>* try to access shared data



Critical Section Goals

- Perhaps they loop (perhaps not!)



Solving the problem

- A first idea:
  - Have a boolean flag, *inside*. Initially false.

Code is <u>unsafe</u>: thread 0 could finish the while test when inside is false, but then 1 might call CSEnter() before 0 can set inside to true!

T0:

```
CSEnter()
    {
        while(inside) continue;
        inside = true;
    }
    #Critical section
CSExit()
    {
        inside = false;
    }
```

T1:

```
CSEnter()
    {
        while(inside) continue;
        inside = true;
    }
    #Critical section
CSExit()
    {
        inside = false;
    }
```

Solving the problem: Take 2

- A different idea (assumes just two threads):
  - Have a boolean flag, *inside[i]*. Initially false.

Code <u>isn't live</u>: with bad luck, both threads could be looping, with 0 looking at 1, and 1 looking at 0

T0:
```
CSEnter(int i)
    {
      ▶ inside[i] = true;
        while(inside[j]) continue;
    }
    #Critical section
CSExit(int i)
    {
        Inside[i] = false;
    }
```

T1:
```
CSEnter(int j)
    {
      ▶ inside[j] = true;
        while(inside[i]) continue;
    }
    #Critical section
CSExit(int j)
    {
        Inside[j] = false;
    }
```

Solving the problem: Take 3

- Another broken solution, for two threads
  - Have a turn variable, *turn*, initially 1.

Code <u>isn't live</u>: thread 1 can't enter unless thread 0 did first, and vice-versa. But perhaps one thread needs to enter many times and the other fewer times, or not at all

T0:

```
CSEnter(int i)
    {
      while(turn != i) continue;
    }
    #Critical section
CSExit(int i)
    {
        turn = j;
    }
```

T1:

```
Other processing…
```

Solving the problem

小结 Three points：

1. 线程是否请求访问该临界资源**（想不想访问）**？

2. 临界资源是否没有任何线程正在访问**（能不能访问）**？

3. 如果临界资源能够访问，如何让所有线程不发生歧义地得知自己接下来能否访问? **（唯一性标识）**

Solving the problem : Take 1

- A first idea:
  - Have a boolean flag, *inside*. Initially false.

Code is <u>unsafe</u>: thread 0 could finish the while test when inside is false, but then 1 might call CSEnter() before 0 can set inside to true!

T0:

```
CSEnter()
    {
        while(inside) continue;
        inside = true;
    }
    #Critical section
CSExit()
    {
        inside = false;
    }
```

唯一性标识

T1:

```
CSEnter()
    {
        while(inside) continue;
        inside = true;
    }
    #Critical section
CSExit()
    {
        inside = false;
    }
```

Solving the problem: Take 2

- A different idea (assumes just two threads):
  - Have a boolean flag, *inside[i]*. Initially false.

Code <u>isn't live</u>: with bad luck, both threads could be looping, with 0 looking at 1, and 1 looking at 0

T0:
```
CSEnter(int i)
    {
      inside[i] = true;
      while(inside[j]) continue;
    }
    #Critical section
CSExit(int i)
    {
      Inside[i] = false;
    }
```

T1:
```
CSEnter(int j)
    {
      inside[j] = true;
      while(inside[i]) continue;
    }
    #Critical section
CSExit(int j)
    {
      Inside[j] = false;
    }
```

能不能访问

Solving the problem: Take 3

- Another broken solution, for two threads
  - Have a turn variable, *turn*, initially 1.

Code <u>isn't live</u>: thread 1 can't enter unless thread 0 did first, and vice-versa. But perhaps one thread needs to enter many times and the other fewer times, or not at all

T0:

```
CSEnter(int i)
    {
      while(turn != i) continue;
    }
    #Critical section
CSExit(int i)
    {
        turn = i ^ 1;
    }
```

T1:

Other processing…

想不想访问

Objectives

- To introduce the critical-section(临界区) problem, whose solutions can be used to ensure the consistency of shared data
- To present both software and hardware solutions of the critical-section problem
- To introduce the concept of an atomic transaction and describe mechanisms to ensure atomicity

Peterson's Solution

- **A classic software-based solution to the critical section problem.**
- **Works for two processes.**
- **The two processes share two variables:**
  - int turn： indicates whose turn it is to enter the critical section. (唯一性标识、能不能访问)
  - Boolean flag[2]： The flag array is used to indicate if a process is ready to enter the critical section. flag[i] = true implies that process P_i is ready! (想不想访问)

Peterson's Algorithm

“礼貌的 Peterson”

```
flag[i] = TRUE; turn = j;
while (flag[j] && turn == j);
  critical section
flag[i] = FALSE;
  remainder section
```

Pi

```
flag[j] = TRUE; turn = i;
while (flag[i] && turn == i);
  critical section
flag[j] = FALSE;
  remainder section
```

Pj

进程 Pi 与 Pj 共享两个参数：

flag[]; (想不想访问)

turn; (唯一性标识；能不能访问)

思考：

如果不是只有两个进程呢？程序中需要改动吗？
如果要改动，改动哪里？

Dekker Algorithm

- Set flag[]=false; turn=0;

```
do{
        flag[i]=true;
        while flag[j]==true{
         if turn !=i{
                flag[i]=false;
                while (turn != i);
                flag[i] =true;
        }
    }
}
```

临界区
turn=1;
flag[i]=false;

“举手放手的Dekker”

Objectives

- To introduce the critical-section(临界区) problem, whose solutions can be used to ensure the consistency of shared data
- To present both software and hardware solutions of the critical-section problem
- To introduce the concept of an atomic transaction and describe mechanisms to ensure atomicity

Synchronization Hardware

- Many systems provide hardware support for critical section code. （许多系统为临界区代码提供硬件支持）
- Hardware features can make programming task easier and improve system efficiency. （优点：简化编程任务，提高系统效率）
- Modern machines provide special atomic hardware instructions to solve critical section problem in a relatively simple manner.
  - Atomic (原子)= non-interruptable
  - 通过特殊的内存访问电路
  - 针对单处理器和多处理器

Solution to Critical-section Problem Using Locks

- In general, any solution to critical-section requires a lock (锁) in order to prevent race condition.

```
do {
        Acquire lock
                #critical section
        Release lock
                #remainder section
} while (true)
```

- Two atomic algorithms:
  - TestAndSet
  - Swep

Solution 1: TestAndSet Instruction

- Definition:

```
boolean TestAndSet (boolean *target)
{
    boolean rv = *target;
    *target = TRUE;
    return rv:
}
```

Solution using TestAndSet

Shared boolean variable lock, initialized to false.

```
while TS(&lock);
  critical section
lock = FALSE;
  remainder section
```

Solution using TestAndSet

Shared boolean variable lock, initialized to false.

```
while (true) {
  while ( TestAndSet (&lock ))
        ;   /* do nothing

  //    critical section

  lock = FALSE;

  //      remainder section

}
```

```
boolean TestAndSet (boolean *target)
 {
   boolean rv = *target;
   *target = TRUE;
   return rv;
 }
```

false

Solution 2: Swap Instruction

- Definition:

```
void Swap (boolean *a, boolean *b)
{
    boolean temp = *a;
    *a = *b;
    *b = temp;
}
```

Solution using Swap

- Shared Boolean variable lock initialized to FALSE; Each process has a local Boolean variable key.

```
key = TRUE;
do
{
   SWAP(&lock, &key);
} while (key);
   critical section
lock = FALSE;
   remainder section
```

Solution using Swap

- Shared Boolean variable lock initialized to FALSE; Each process has a local Boolean variable key.
- Solution:

```
while (true)  {
        key = TRUE;
        while ( key == TRUE)
                Swap (&lock, &key );

        //    critical section

          lock = FALSE;

          //      remainder section
    }
```

忙等待和无忙等待

- 忙等待

```
AcquireLoc(){

while(TestandSet(&lock))
}
临界区
Release(){
        *lock=false;
}
```

忙等待和无忙等待

- 无忙等待

```
AcquireLoc(){
        while(TestandSet(&lock))
            add this TCB to wait queue q;
}
临界区
Release() (){
        *lock=false;
        remove one thread t from q;
  wakeup(t)
}
```

原子操作优点

- 适用于单处理器或者共享主存的多处理器中任意数量的进程。
- 简单并且容易证明
- 可以用于支持多临界区

原子操作的缺点

- 忙等待消耗处理器时间
- 当进程离开临界区并且多个进程在等待的时候可能导致饥饿
- 死锁：
  - 如果一个低优先级的进程拥有临界区，但同时一个高优先级进程也有需求，那么高优先级进程会获得处理器，但不得不等待临界区

禁用中断

- 没有中断，没有上下文切换，因此没有并发
  - 硬件将中断处理延迟到中断被启用之后
  - 大多数现代计算机体系结构都提供指令来完成
- 进入临界区
  - 禁用中断
- 离开临界区
  - 开启中断

禁用中断缺点

- 一旦中断被禁用，线程就无法被停止
  - 整个系统都会为该线程停下来
  - 可能导致其他线程处于饥饿状态
- 如果临界区可以任意长：
  - 无法限制响应中断所需的时间
- 只能禁用该 CPU 自己的中断指令，无法控制其他 CPU 的中断指令，仍然存在隐患

An example

- 当队列中有 n 个进程 $p_0, p_2, \ldots p_{n-1}$, 如何管理他们的临界区？用原语写出一段访问临界资源的代码。

```
do {
        waiting[i]=true;  ( 想不想访问 )
        key = true;  ( 唯一性标识 )
      while (waiting[i] && key)
          key=TestAndSet (&lock);
                                  ( 能不能访问 )
        //    critical section

      j=(i+1) % n;
        while((j != i)) && !waiting[j]
                j=(j+1) % n;

      if(j == i)
                lock=false;
        else
                waiting[j]=false;

}while(true)
```

临界区小结

- 锁是解决临界区问题的常用方法：
  - 互斥可以使用锁来实现
  - 通常需要一定等级的硬件支持
- 常用的三种实现方法：
  - 禁用中断（仅限于单处理器）
  - 软件方法（复杂）
  - 硬件方法（原子操作指令，单处理器或多处理器均可）
- 可选的实现内容：
  - 有忙等待
  - 无忙等待

本节小结

- **Background (race condition，竞争条件)**
- **The Critical-Section Problem (临界区问题)**
- **Peterson's Solution (software-based)**
- **Synchronization Hardware (hardware-based: automic)**
- Semaphores
- Classic Problems of Synchronization
- Monitors
- Synchronization Examples
- Atomic Transactions

习题

- 1． 操作系统中有一组常称为特殊系统调用的程序，它不能被系统中断，在操作系统中称为（）

A 初始化程序

B 原语

C 子程序

D 控制模块

习题

- 2． 在一段时间内，只允许一个进程访问的资源称为（）

A 共享资源

B 临界区

C 临界资源

D 共享区

习题

- 3． 临界区是指并发进程中访问共享变量的（）段

A 管理信息

B 信息存储

C 程序

D 数据

例题

【多选】在单处理机系统中，设系统中有 n 个进程（n>2），且当前处理机没有执行进程调度程序，下述情况哪些可能发生（ ）。

A. 没有运行的进程，有 2 个进程处于就绪状态， n 个进程处于等待状态。

B. 一个进程处于运行状态， n-1 个进程处于等待状态

C. 一个进程处于运行状态， 1 个进程处于就绪状态，n-2 个进程处于等待状态。

D. 一个进程处于运行状态， n-1 个进程处于就绪状态，没有进程处于等待状态

答案： BCD

Chapter 6: Process Synchronization

- Background
- The Critical-Section Problem
- Peterson's Solution
- Synchronization Hardware
- Semaphores
- Classic Problems of Synchronization
- Monitors
- Synchronization Examples
- Atomic Transactions

Semaphore

- *A* Semaphore is a special integer variable （特殊的整数变量）, it can be accessed only through two standard atomic operations:
  - Wait() and Signal()
  - Originally called P() and V()
  - Wait(S) {

```
        while S <= 0
                ; // busy wait
        S--;
        }
```

  - signal (S) {

```
      S++;
    }
```

Semaphore Implementation

- The main disadvantage of the above semaphore definition is busy waiting which wastes CPU cycles.
  - Spinlock （自旋锁）
  - Process spins while waiting for the lock.
  - In a multiprocessor system, spinlock is often used to protect a short code. （通常用于保护短代码）
- Note that applications may spend lots of time in critical sections and therefore this is not a good solution. （不是一个好的解决方案）

Semaphore Implementation with no Busy waiting

- To overcome busy waiting, we can modify the implementation of semaphore S and the definition of wait() and signal(). （修改 wait() 和 signal() 的定义）
- Now that a semaphore is a struct.
  - a value (of type integer)
  - a waiting queue
- Two operations:
  - block – place the process invoking the operation on the appropriate waiting queue.
  - wakeup – remove one of processes in the waiting queue and place it in the ready queue.

Semaphore Implementation with no Busy waiting (Cont.)

- Implementation of wait:

```
wait (S){
        value--;
        if (value < 0) {
                add this process to waiting queue
                block();  }
}
```

- Implementation of signal:

```
Signal (S){
        value++;
        if (value <= 0) {
                remove a process P from the waiting queue
                wakeup(P);  }
}
```

Semaphore as General Synchronization Tool

- Counting semaphore
  - integer value can range over an unrestricted domain
  - Can be used to control access to a given resource consisting of a finite number of instances.
- Binary semaphore
  - integer value can range only between 0 and 1; can be simpler to implement
  - Also known as mutex locks （互斥锁）
  - Provides mutual exclusion
  - Semaphore S; // initialized to 1
  - wait (S);

    Critical Section

    signal (S);

Deadlock and Starvation

- The implementation of a semaphore with a waiting queue may result in one of the two following situations. （导致死锁和饥饿）
- Deadlock （死锁） – two or more processes are waiting indefinitely for an event that can be caused by only one of the waiting processes
- Let S and Q be two semaphores initialized to 1

P_0	P_1
wait (S);	wait (Q);
wait (Q);	wait (S);
.	.
.	.
.	.
signal (S);	signal (Q);
signal (Q);	signal (S);

- Starvation （饥饿） – indefinite blocking. A process may never be removed from the semaphore queue in which it is suspended.

Deadlock and Starvation

- The implementation of a semaphore with a waiting queue may result in one of the two following situations.
- Deadlock – two or more processes are waiting indefinitely for an event that can be caused by only one of the waiting processes
- Let S and Q be two semaphores initialized to 1

P_0	P_1
wait (S);	wait (Q);
wait (Q);	wait (S);
.	.
.	.
.	.
signal (S);	signal (Q);
signal (Q);	signal (S);

Deadlock and Starvation

- The implementation of a semaphore with a waiting queue may result in one of the two following situations.
- Deadlock – two or more processes are waiting indefinitely for an event that can be caused by only one of the waiting processes
- Let S and Q be two semaphores initialized to 1

P_0	P_1
wait (S);	wait (Q);
wait (Q);	wait (S);
.	.
.	.
.	.
signal (S);	signal (Q);
signal (Q);	signal (S);

Deadlock and Starvation

- The implementation of a semaphore with a waiting queue may result in one of the two following situations.
- Deadlock – two or more processes are waiting indefinitely for an event that can be caused by only one of the waiting processes
- Let S and Q be two semaphores initialized to 1

P_0	P_1
wait (S);	wait (Q);
wait (Q);	wait (S);
.	.
.	.
.	.
signal (S);	signal (Q);
signal (Q);	signal (S);

- Starvation – indefinite blocking. A process may never be removed from the semaphore queue in which it is suspended.

AND Semaphore

```
Swait(S1, S2, ..., Sn)
          {
while (TRUE)
{
    if (S1 >=1 && S2 >= 1 && ... && Sn >= 1)
    {
      for (i = 1; i <= n; ++i)  --Si;

      break;
    }
 else
 {   add the process to Sj.queue //no available resource to use
and Sj is the first semaphore of which value is less than 1 ;
      block();
    }
}
}
```

AND Semaphore

```
Ssignal(S1, S2, ..., Sn)
{
  for (i = 1; i <= n; ++i)
  {
    ++Si;  //release resources ;
    for (each process P waiting in Si.queue)
        //check if there is some process waiting ;
    {
      remove a process P from Si.queue;
      if (Swait(S1, S2, ..., Sn) )
         //different from  signal, P will be waken up when it
acquires all resources  ;
        {  //all needed resources are available ;
             wakeup(P);
        }
      else
        {  //otherwise ;
             add process P to another waiting queue ;
        }
    }
  }
```

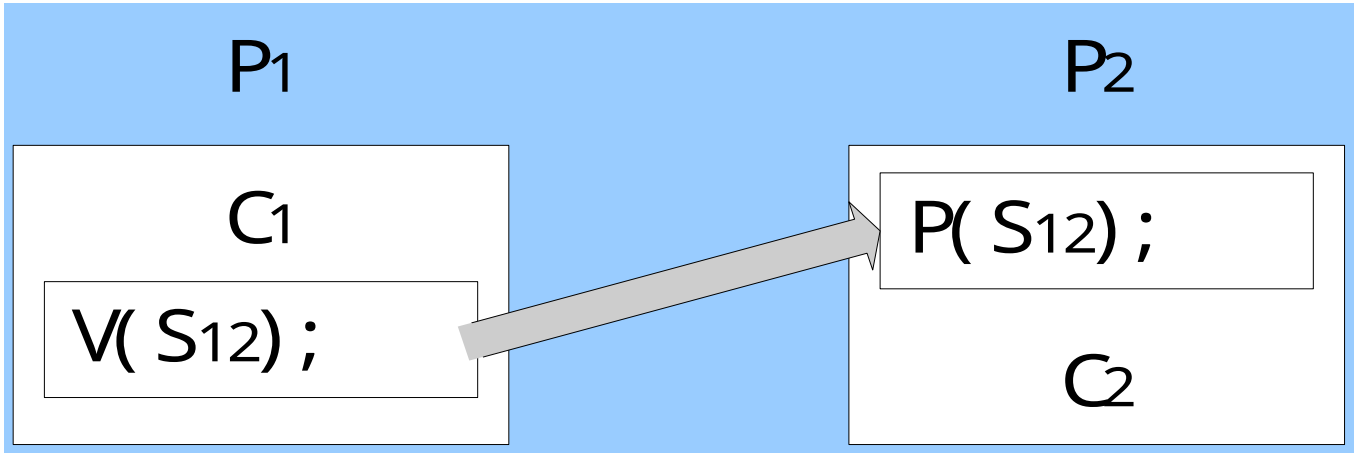
（互斥） Mutual exclusion using semaphore

Semaphore mutex ; // initialized to 1

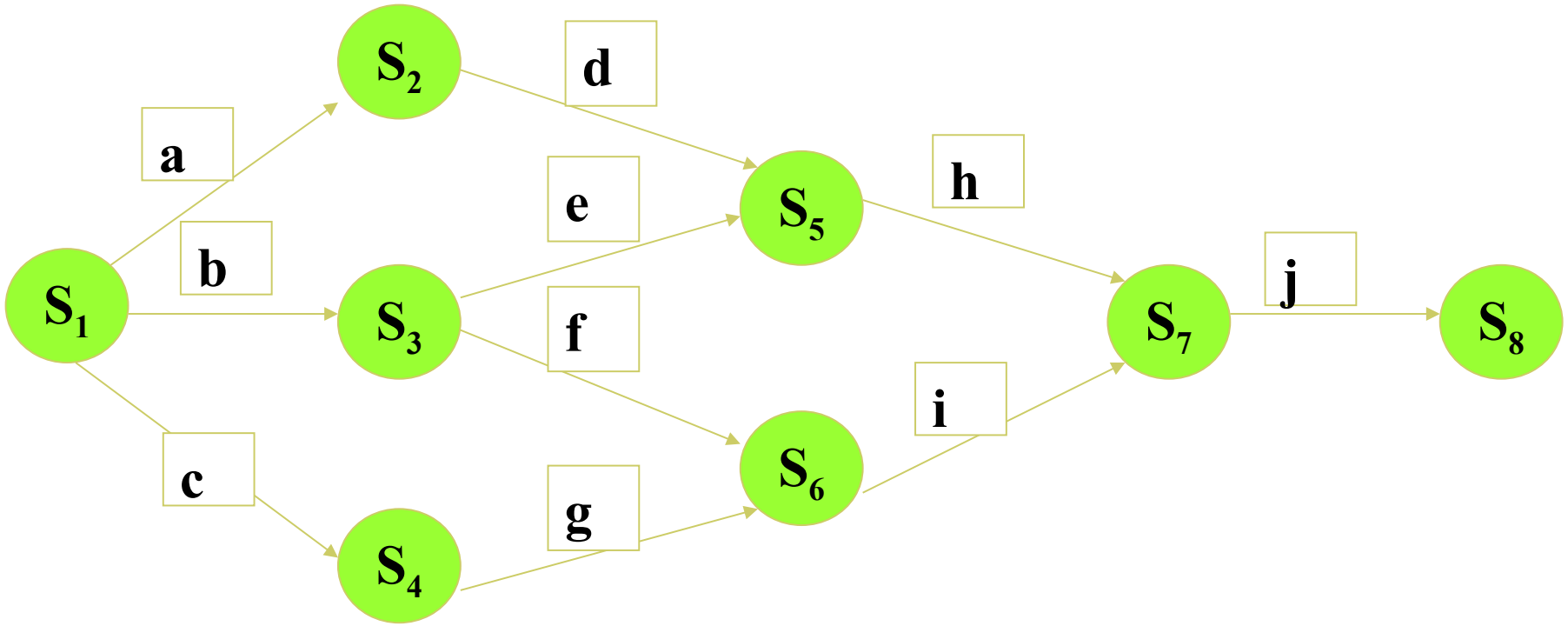
```
P(mutex);
    critical section
V(mutex);
    remainder section
```

（同步） Synchronization using semaphore

- Consider two concurrently running processes P1 and P2, P1 with a statement C1 and P2 with a statement C2. suppose we require that C2 be executed only after C1 has completed
- We can define a common semaphore S12, initialized to 0.



Example 1



Struct

smaphore a,b,c,d,e,f,g,h,I,j=0,0,0,0,0,0,0,0,0,0

cobegin

{S1;V(a);V(b);V(c);}

{P(a);S2;V(d);}

{P(b);S3;V(e);V(f);}

{P(c);S4;V(g);}

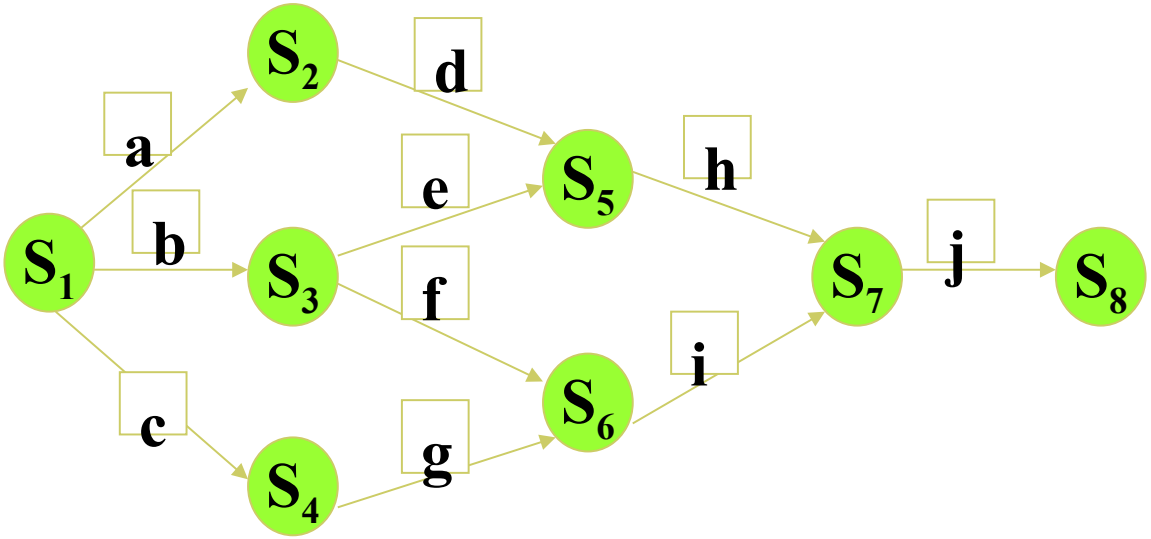
{P(d);P(e);S5;V(h);}

{P(f);P(g);S6;V(i)}

{P(h);P(i);S7;V(j);}

{P(j);S8;}

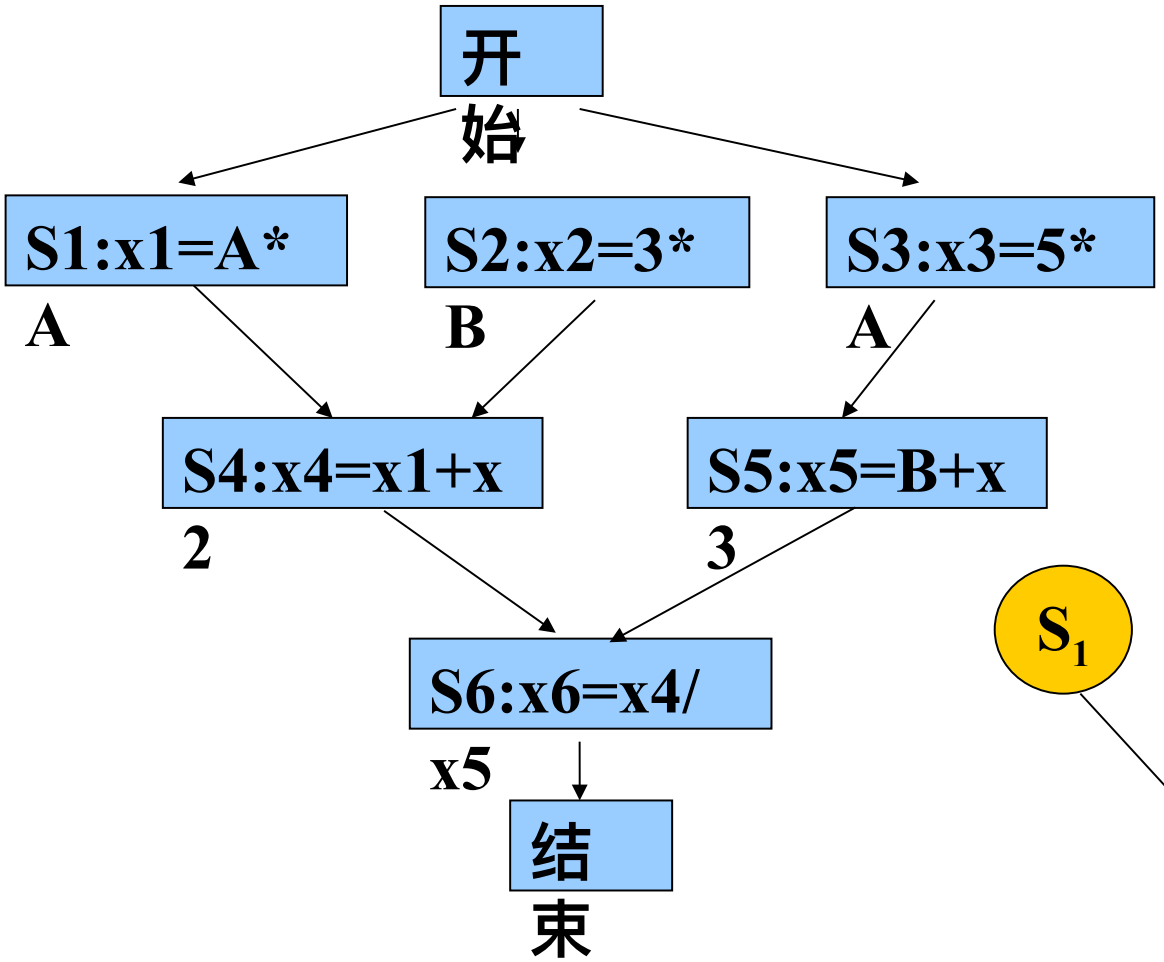
coend

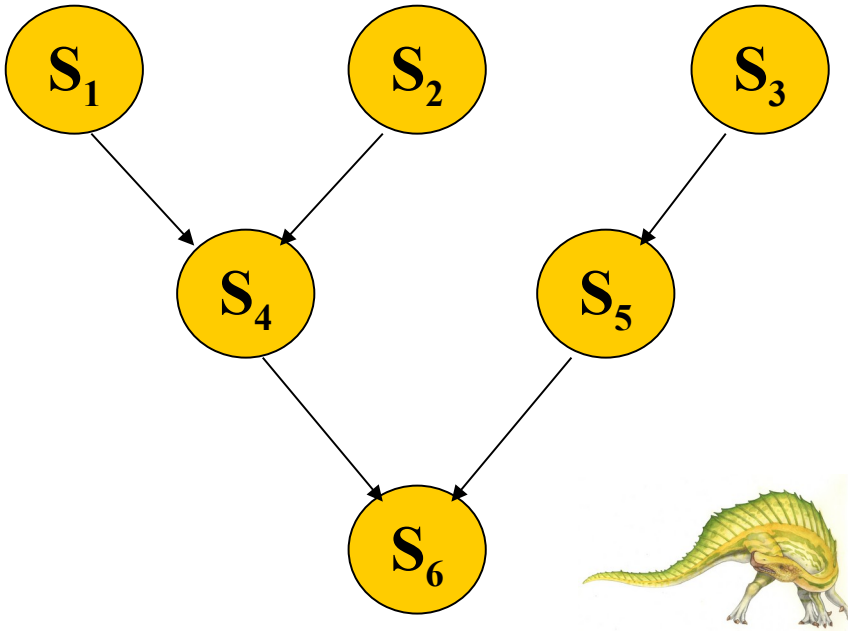


Example 2

- （A^2+3B） /(B+5A), 若 A,B 已赋值,

试画出该公式求值过程的前趋图。


开始
S1:x1=A*
A
S2:x2=3*
B
S3:x3=5*
A
S4:x4=x1+x
2
S5:x5=B+x
3
S6:x6=x4/
x5
结束


S_1
S_2
S_3
S_4
S_5
S_6

$(A^2+3B)/(B+5A)$

Chapter 6: Process Synchronization

- Background
- The Critical-Section Problem
- Peterson's Solution
- Synchronization Hardware
- Semaphores
- Classic Problems of Synchronization
- Monitors
- Synchronization Examples
- Atomic Transactions

Classical Problems of Synchronization

- Bounded-Buffer Problem （有界缓冲问题）
- Readers and Writers Problem （读者 - 作者问题）
- Dining-Philosophers Problem （哲学家就餐问题）

Bounded-Buffer Problem

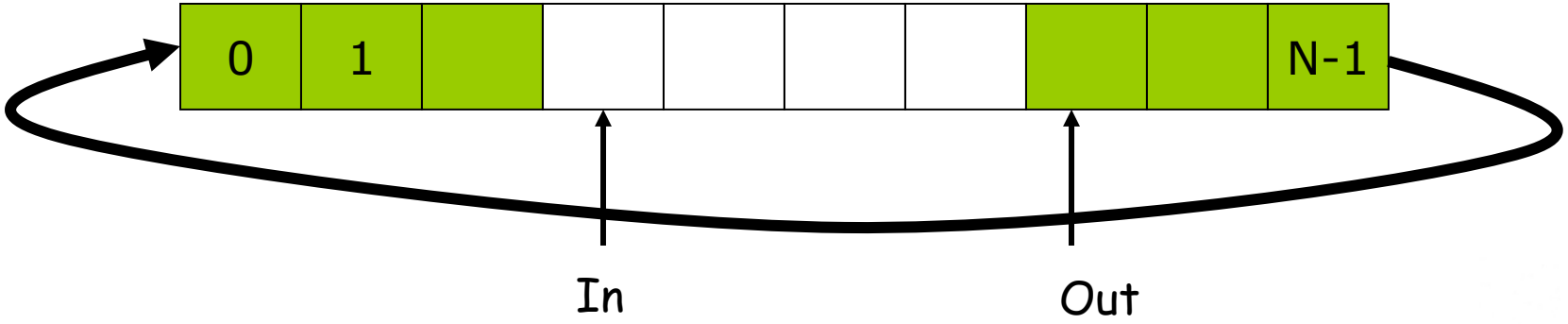
- Problem:
  - producer puts things into a shared buffer
  - Consumer takes them out
  - Need synchronization for coordinating producer and consumer

Bounded-Buffer Problem

- Coke machine example:
  - delivery person (producer) fills machine with cokes
  - students (consumer) feed cokes and drink them
  - coke machine has finite space (buffer)

Producer-Consumer Problem

- Bounded buffer: size ‘N’
  - Access entry 0… N-1, then “wrap around” to 0 again
- Producer process writes data to buffer
  - Must not write more than ‘N’ items more than consumer “ate”
- Consumer process reads data from buffer
  - Should not try to consume if there is no data



Bounded-Buffer Problem

- *N* buffers, each can hold one item
- Semaphore mutex initialized to the value 1
- Semaphore full initialized to the value 0
- Semaphore empty initialized to the value N.

The structure of Producer/Consumer process

Producer	Consumer
P(empty); P(mutex); //进入区	P(full); P(mutex); //进入区
one unit --> buffer;	one unit <-- buffer;
V(mutex); V(full); //退出区	V(mutex); V(empty); //退出区

Bounded-Buffer Problem

```
public class BoundedBuffer  {
    public BoundedBuffer() { /* see next slides */  }
    public void enter() { /* see next slides */  }
    public Object remove() { /* see next slides */  }

    private static final int   BUFFER_SIZE = 2;
    private Semaphore mutex;
    private Semaphore empty;
    private Semaphore full;
    private int count, in, out;
    private Object[] buffer;
}
```

Bounded Buffer Constructor

```
public BoundedBuffer() {
    // buffer is initially empty
    count = 0;
    in = 0;
    out = 0;
    buffer = new Object[BUFFER_SIZE];
    mutex = new Semaphore(1);
    empty = new Semaphore(BUFFER_SIZE);
    full = new Semaphore(0);
}
```

enter() Method

```
public void enter(Object item) {
    empty.P();
    mutex.P();
    // add an item to the buffer
    ++count;
    buffer[in] = item;
    in = (in + 1) % BUFFER_SIZE;
    mutex.V();
    full.V();
}
```

remove() Method

```
public Object remove() {
    full.P();
    mutex.P();
    // remove an item from the buffer
    - - count;
    Object item = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
    mutex.V();
    empty.V();
    return item;
}
```

What's wrong?

```
Init: Semaphore mutex = 1;    /* for mutual exclusion*/
      Semaphore empty = N;  /* number empty buf entries */
      Semaphore full = 0;      /* number full buf entries */
      any_t buf[N];
      int t
```

Oops! Even if you do the correct operations, the *order* in which you do semaphore operations can have an incredible impact on correctness

Producer

```
void put(char ch) {

   wait(mutex);
   wait (empty);

   // add ch to buffer
   buf[head%N] = ch;
   head++;

   signal(mutex);
   signal (full );
}
```

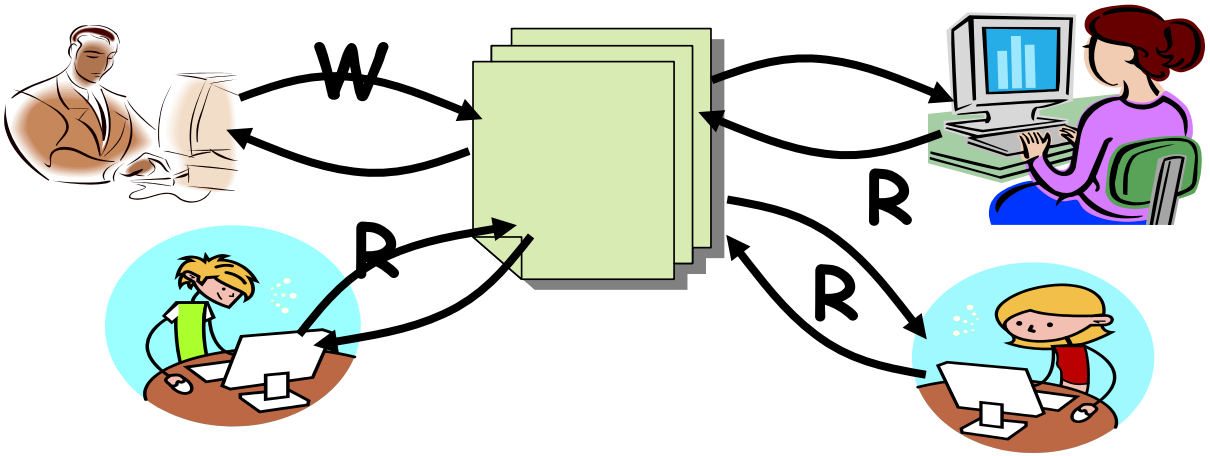
What if buffer is full?

```
   wait (full );
   wait (mutex);

   // remove ch from buffer
   ch = buf[tail%N];
   tail++;

   signal (mutex);
   signal (empty);
   return ch;
}
```

Discussion about Bounded Buffer Solution

- Why asymmetry （不对称） ?
  - Producer does: wait (empty), signal(full)
  - Consumer does: wait (full), signal(empty)
- Is order of wait ’s important?
  - Yes! Can cause deadlock
- Is order of signal’s important?
  - No, except that it might affect scheduling efficiency
- What if we have 2 producers or 2 consumers?
  - Do we need to change anything?

Readers-Writers Problem



- Motivation: Consider a shared database
  - Two classes of users:
    - Readers – never modify database
    - Writers – read and modify database

Readers-Writers Problem

- Problem – allow multiple readers to read at the same time. Only one single writer can access the shared data at the same time.
- Shared Data
  - Data set
  - Semaphore Rmutex initialized to 1.
  - Semaphore Wmutex initialized to 1.
  - Integer readcount initialized to 0.

Readers-Writers Problem

Writer	Reader
P(Wrmutex); write; V(Wrmutex);	P(Rmutex); if (Rcount == 0) P(Wrmutex); ++Rcount; V(Rmutex); read; **P(Rmutex);** --Rcount; if (Rcount == 0) V(Wrmutex); **V(Rmutex);**

Readers-Writers Problem: Reader

```
public class Reader extends Thread {
   public Reader(Database db) {
                server = db;
   }
   public void run() {
     int c;
     while (true) {
                c = server.startRead();
                // now reading the database
                c = server.endRead();
     }
   }
   private Database server;
}
```

Readers-Writers Problem (cont)

```
public class Database
{
   public Database() {
      readerCount = 0;
      mutex = new Semaphore(1);
      db = new Semaphore(1);
   }
      public int startRead() { /* see next slides */ }
      public int endRead()  { /* see next slides */ }
      public void startWrite() { /* see next slides */ }
      public void endWrite()  { /* see next slides */ }

      private int readerCount; // number of active readers
      Semaphore mutex;   // controls access to readerCount
      Semaphore db;      // controls access to the database
}
```

startRead() Method

```
public int startRead() {
    mutex.P();
    ++readerCount;

    // if I am the first reader tell all others
    // that the database is being read
    if (readerCount == 1)
      db.P();

    mutex.V();
    return readerCount;
  }
```

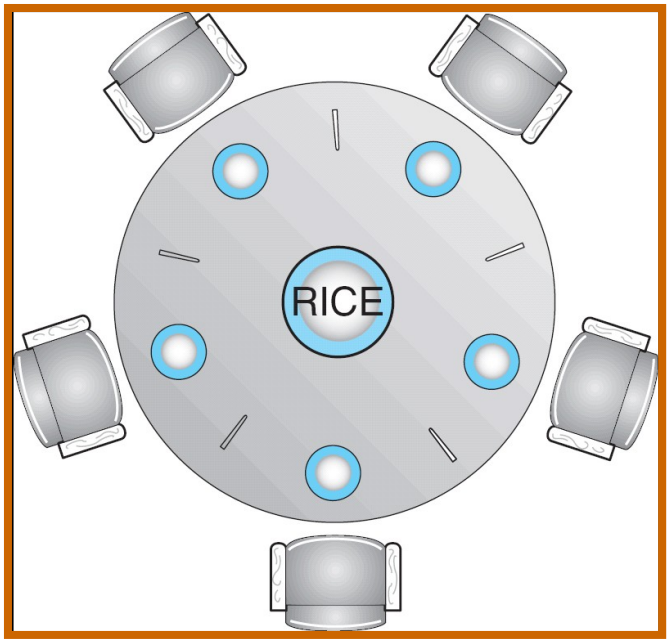
endRead() Method

```
public int endRead() {
    mutex.P();
    --readerCount;

    // if I am the last reader tell all others
    // that the database is no longer being read
    if (readerCount == 0)
      db.V();

    mutex.V();
    return readerCount;
  }
```

Writer Methods

```
public void startWrite() {
    db.P();
}

public void endWrite() {
    db.V();
}
```

Dining-Philosophers Problem



- Shared data
  - Bowl of rice (data set)
  - Semaphore chopstick [5] initialized to 1

Dining-Philosophers Problem (Cont.)

- The structure of Philosopher *i*:

```
While (true)  {
        wait ( chopstick[i] );
        wait ( chopStick[ (i + 1) % 5] );

                //  eat

        signal ( chopstick[i] );
        signal (chopstick[ (i + 1) % 5] );
                //  think
}
```

Deadlock?

Solution2： AND semaphore

```
Var chopstick array[0,…,4] of semaphore :=(1,1,1,1,1);
Processi
   Reeat
         think;
         Swait (chopstick[(I+1) mod 5],chopstick[I]);
         Eat;
         Ssignal (chopstick[(I+1) mod 5],chopstick[I]);
 Until false;
```

Discussion about Semaphores

- The value of a semaphore represents the number of available resource
  - S>0: there are S available resources
  - S=0: all resources are being used
  - S<0: there are |S| processes waiting for the resource
- Operations can be done to a semaphore S:
  - Initialization: >=0
  - Wait(): request a resource
  - signal (): release a resource

Discussion about Semaphores(Cont.)

- Either wait(S) or signal(S) can not be omitted
  - For **mutual exclusion**: they appear in the same process （互斥在一边）
  - For **synchronization**: they appear in different process respectively （同步分两边）
- For two successive wait(S1) and wait(S2) in a process, the order is important. If S1 is for synchronization, and S2 is for mutual exclusion, then
  - The correct order is wait(S1) ; wait(S2)
  - If they are written in the order wait(S2) ;wait(S1) , there would be a deadlock.

Tips

- 互斥 / 同步的对象是谁?
- 互斥访问什么?
- 同步什么内容?

例题

- 进程之间存在哪几种相互制约关系？各是什么原因引起的？下列活动分别属于哪种制约关系？
- （1）若干同学去图书馆借书
- （2）流水线生产的各道工序
- （3）商品生产和社会消费

答案：

（1）互斥：图书馆的书；

（2）同步：各道工序间同步

（3）同步：先生产，再消费；

例题：工厂分配问题

工厂装配问题

- 某工厂有两个生产车间和一个装配车间，两个生产车间分别生产 A， B 两种零件，装配车间的任务是把 A,B 两种零件组装成产品。两个生产车间每生产一个零件后，都要分别把它们送到专配车间的货架 F, F2 上。 F1 存放零件 A， F2 存放零件 B， F1 和 F2 的容量均可存放 10 个零件。装配工人每次从货架上取一个零件 A 和一个零件 B 后组装成产品。请用 P,V 操作进行正确管理。

例题：工厂分配问题

分析：哲学家问题 + 生产者消费者问题的变种

- 互斥 / 同步的对象是谁?
  - 生产者：生产车间 A 、生产车间 B
  - 消费者：装配车间
- 互斥访问什么?
  - 货架 F1 、 F2
- 同步什么内容?
  - 先生产，再装配
    - 货架 F1 上零件 A 的数量
    - 货架 F2 上零件 B 的数量
- 解答：设置 6 个信号量：
  - m1=1, m2=1 实现货架 F1 、 F2 的互斥使用
  - empty_1=10, full_1=0 表示货架 F1 上零件 A 的数量情况
  - empty_2=10, full_2=0 表示货架 F2 上零件 B 的数量情况

例题：工厂分配问题

- 生产车间 A：
  - P （empty_1）
  - P(m1)
  - 将 A 零件放到货架 F1 上；
  - V(m1)
  - V(full_1)
- 生产车间 B：
  - P （empty_2）
  - P(m2)
  - 将 B 零件放到货架 F1 上；
  - V(m2)
  - V(full_2)
- 装配车间：
  - P （ full_1 ）
  - P （ full_2）
  - P(m1) &P(m2)
  - 将 A 、 B 零件组装起来；
  - V(m1)
  - V(m2)
  - V(full_2)
  - V(full_1)

例题：吃水果问题

吃水果问题

- 桌上有一只盘子，每次只能放一个水果，爸爸向盘中放苹果或桔子，儿子专等吃盘里的桔子，女儿专等吃盘里的苹果。只要盘子空，则爸爸可向盘中放水果，仅当盘中有自己需要的水果时，儿子或女儿可从中取出 .
- 请给出三人之间的同步关系，并用 P 、 V 操作实现三人正确活动的程序。

例题：吃水果问题

- 分析 : 此问题实际上是生产者 - 消费者问题的一个变形
- 互斥 / 同步的对象是谁?
  - 生产者： 爸爸
  - 消费者：女儿、儿子
- 互斥访问什么?
  - 缓冲区：盘子
- 同步什么内容?
  - 苹果个数、桔子个数
- 解答：设置 3 个信号量：
  - mutex=1, 实现盘子的互斥使用
  - empty=1, apple=0,orange=0; 盘子里空位个数；苹果个数；桔子个数

例题：吃水果问题

- 爸爸：
  - P （empty)
  - P(mutex)
  - 将水果放入盘中；
  - IF 桔子 V （orange）
  - else V （apple）
  - V(mutex)

儿子： P(orange)

P(mutex)

从盘中取桔子；

V(mutex)

V （ empty ）

吃桔子

女儿： P (apple)

P(mutex)

从盘中取苹果；

V(mutex)

V （ empty ）

吃苹果

Chapter 6: Process Synchronization

- Background
- The Critical-Section Problem
- Peterson's Solution
- Synchronization Hardware
- Semaphores
- Classic Problems of Synchronization
- Monitors
- Synchronization Examples
- Atomic Transactions

Monitors

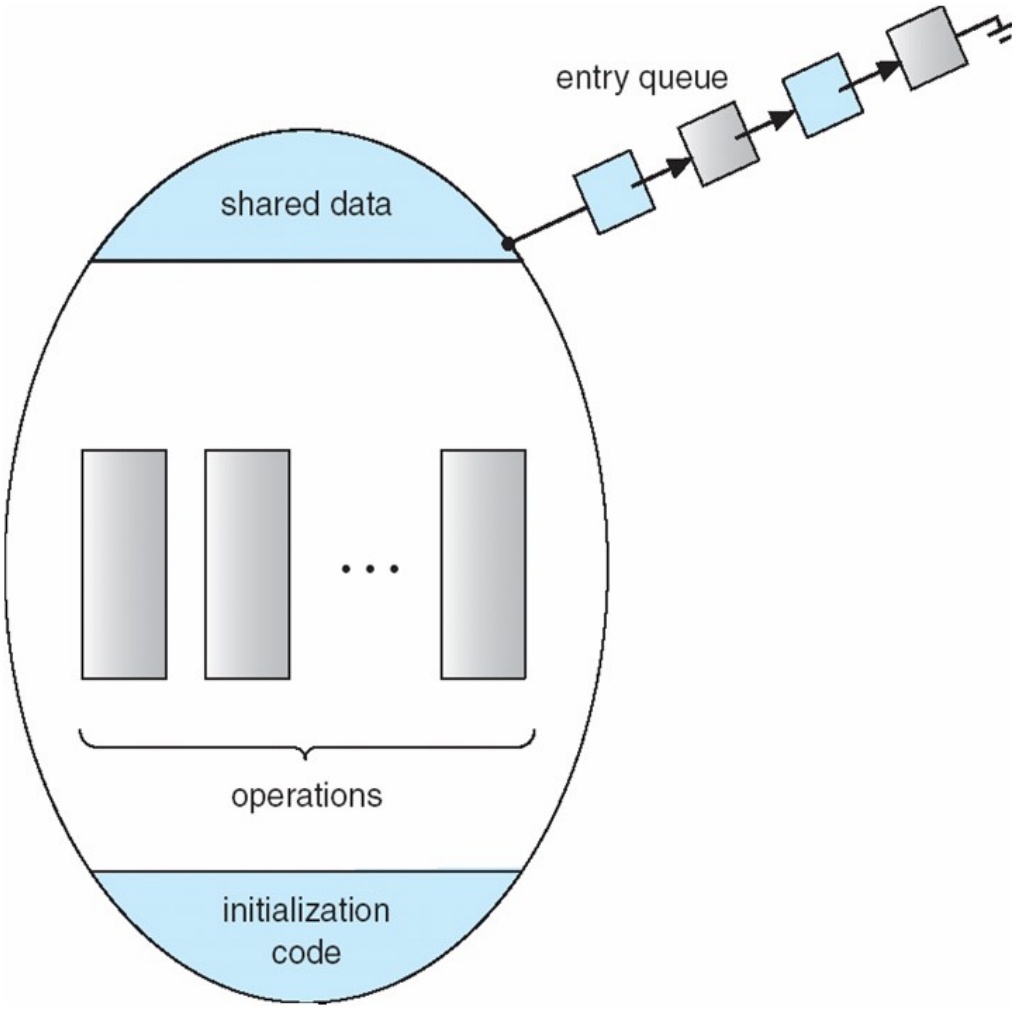
- A high-level abstraction that provides a convenient and effective mechanism for process synchronization
- Only one process may be active within the monitor at a time

```
monitor monitor-name
{
   // shared variable declarations
   procedure P1 (…) { …. }
            …

   procedure Pn (…) {……}

   Initialization code ( ….) { … }
            …
   }
}
```

Monitors

- 管程定义：一个管程定义了一个数据结构和能为并发进程调用的在该数据结构上的一组操作过程，这组互斥操作的过程，能同步进程和改变管程中的数据。
- Access to it is guaranteed to be exclusive. （对它的访问保证是独占的）
- Monitors use separate mechanisms for the two types of synchronization
  - use locks for mutual exclusion （使用锁实现互斥）
  - use condition variables for ordering constraints （使用条件变量实现顺序执行）
- **monitor = a lock + the condition variables associated with that lock**

Schematic view of a Monitor



Condition Variables

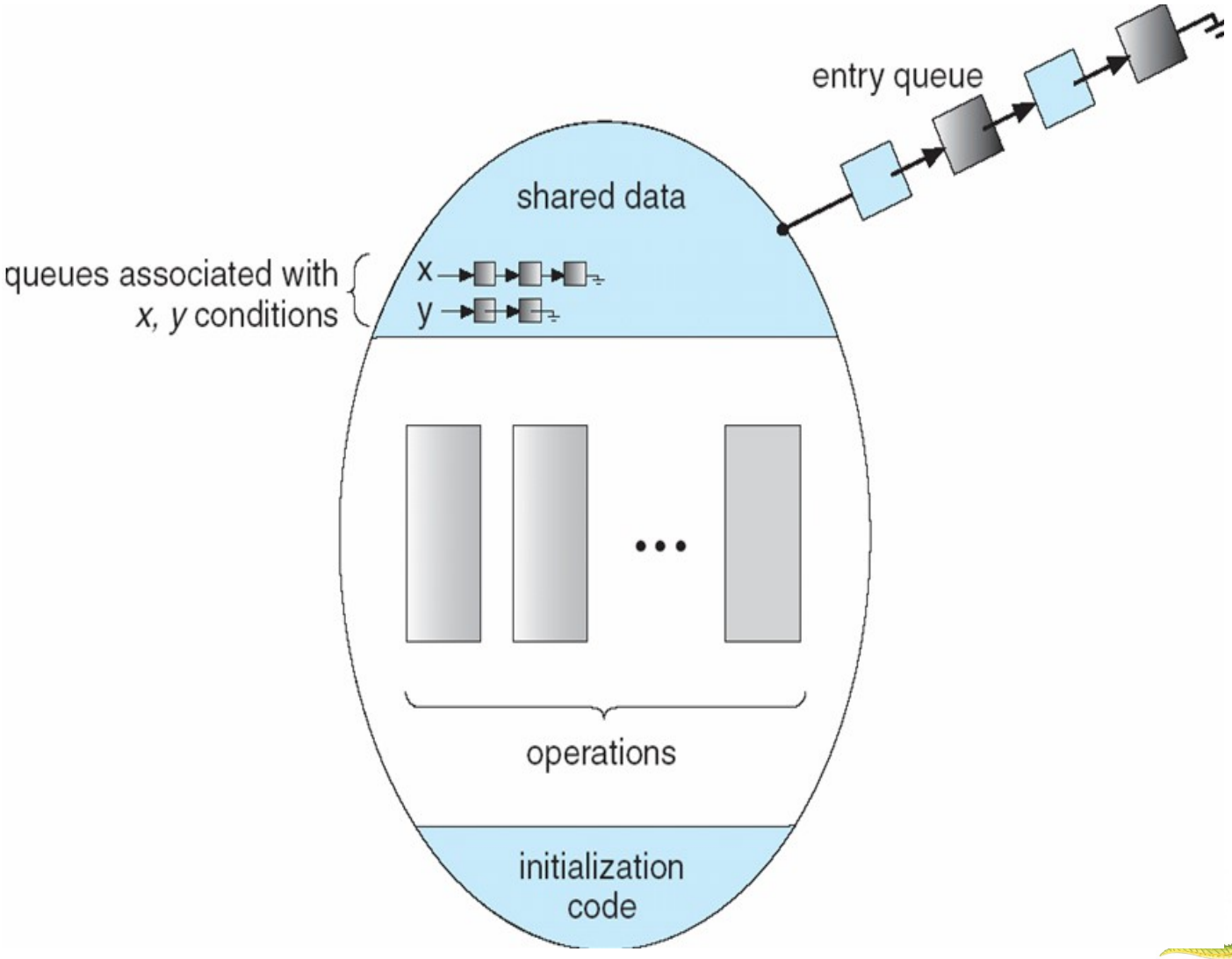
- Main idea:
  - make it possible for thread to sleep inside a critical section
- Approach:
  - by atomically releasing lock, putting thread on wait queue and go to sleep
- Each variable has a queue of waiting threads
  - threads that are sleeping, waiting for a condition

Condition Variables

- condition x, y;

- Two operations on a condition variable:
  - x.wait () – a process that invokes the operation is suspended.
  - x.signal () – resumes one of processes (if any) that invoked x.wait ()

Monitor with Condition Variables



Producer Consumer using Monitors

```
Monitor Producer_Consumer {
    any_t buf[N];
    int n = 0, tail = 0, head = 0;
    condition full, empty;
    void put(char ch) {
        if(n == N)  empty.wait();
        buf[head%N] = ch;
        head++;
        n++;
        full.signal();
    }
    char get()  {
        if(n == 0)   full.wait();
        ch = buf[tail%N];
        tail++;
        n--;
        empty.signal();
        return ch;
    }
}
```

What if no thread is waiting when signal is called?

Signal is a "no-op" if nobody is waiting. This is very different from what happens when you call signal () on a semaphore – semaphores have a "memory" of how many times signal() was called!
(信号量记录了有多少个需求没被满足)

Producer Consumer using Monitors

```
producer : begin
                repeat
                    produce an  item  in nextp ;
                    Producer_Consumer.put ( item) ;
                until false   ;
            end
consumer : begin
                repeat
                    Producer_Consumer.get (item) ;
                    consume the item in nextc ;
                until false ;
            end
```

Solution to Dining Philosophers

```
monitor DP
  {
    enum { THINKING; HUNGRY, EATING } state [5] ;
    condition self [5];

    void pickup (int i) {
          state[i] = HUNGRY;
          test(i);
          if (state[i] != EATING) self [i].wait;
    }

     void putdown (int i) {
          state[i] = THINKING;
                // test left and right neighbors
           test((i + 4) % 5);
           test((i + 1) % 5);
      }
```

Solution to Dining Philosophers (cont)

```
    void test (int i) {
            if ( (state[(i + 4) % 5] != EATING) &&
            (state[i] == HUNGRY) &&
            (state[(i + 1) % 5] != EATING) ) {
                state[i] = EATING ;
                    self[i].signal () ;
             }
     }

      initialization_code() {
           for (int i = 0; i < 5; i++)
           state[i] = THINKING;
    }
}
```

Solution to Dining Philosophers (cont)

- Each philosopher *I* invokes the operations pickup() and putdown() in the following sequence:

dp.pickup (i)

EAT

dp.putdown (i)

Monitor Implementation Using Semaphores

- For a monitor, a semaphore mutex(initially = 1) is necessary to ensure mutual exclusion.
  - A process must execute wait(mutex) before entering the monitor, and execute signal(mutex) after leaving the monitor.
- Another semaphore next(initially =0) is introduced for a signaling process to suspend itself until the signaled process leaves or waits.
- Next-count is an integer variable to count the number of processes suspended on next.(有多少在等 next)

Monitor Implementation Using Semaphores

- Variables

```
semaphore mutex;  // (initially  = 1)
semaphore next;   // (initially  = 0)
int next-count = 0;
```

- Each procedure ***F*** will be replaced by

```
wait(mutex);
    …
                    body of F;

    …
if (next-count > 0)
        signal(next)
else
        signal(mutex);
```

- Mutual exclusion within a monitor is ensured.

例题

如果有三个进程共享同一互斥段，而且每次最多允许两个进程进入该互斥段，则信号量的初值应设置为（ ）。

A. 3　　B. 1　　C. 2　　D. 0

答案：　C

例题

（　　）操作不是 P 操作可完成的。

A. 为进程分配处理机　　B. 使信号量的值变小

C. 可用于进程的同步　　D. 使进程进入阻塞状态

答案：　A

例题

【多选】进程从执行状态转换到阻塞状态的可能原因是 ().

A. 时间片完

B. 需要等待其它进程的执行结果

C. 执行了 V 操作

D. 执行了 P 操作

答案： BD

例题

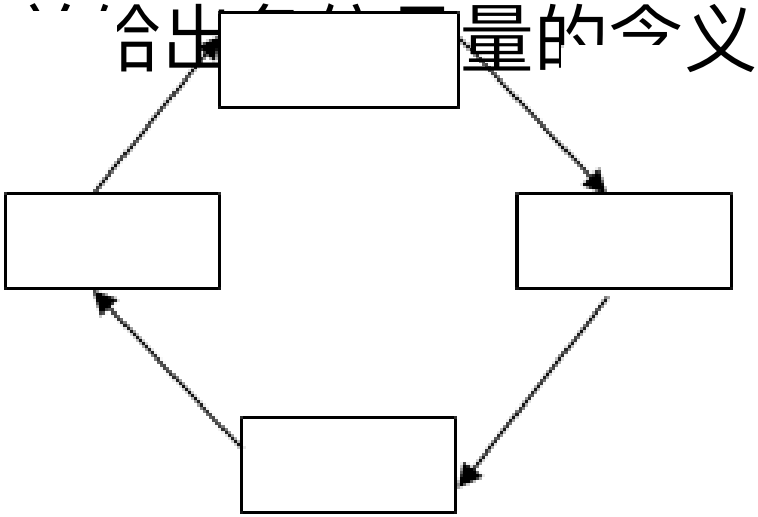
【解答题】设有两个优先级相同的进程 P1 和 P2 如下。信号量 S1 和 S2 的初值均为 0，试问 P1 、 P2 并发执行结束后， x=?， y=?， z=?

〈进程 P1 〉	〈进程 P2 〉
y:=1;	x:=1;
y:=y+2;	x:=x+1;
V(S1);	P(S1);
z:=y+1;	x:=x+y;
P(S2) ;	V(S2) ;
y:=z+y;	z:=x+z;

答案： x=5， y=12， z=9 或 x=5， y=7， z=9

例题

【解答题】已知系统中有四个缓冲池 M1， M2，M3， M4。其大小分别为 3、2、3、2，各缓冲池分别存在 0， 1， 0， 2 个缓冲区有数据。现同时有四个进程 P1， P2， P3， P4 分别在各缓冲区间不断地移动数据。见下图。例如， P1 进程从 M1 向 M2 移动数据。试用信号量及其 P、V （或 signal,wait ）操作及类 Pasic/C 语言描述各进程之间的同步关系，给出各信号量的含义和初值。



Tips

- 互斥 / 同步的对象是谁?
- 互斥访问什么?
- 同步什么内容?

例题

答案：

（1）互斥信号量和初值

m(1)=1,m(2)=1,m(3)=1,m(4)=1

（2）同步信号量

full(i) 表示 buffer(i) 是否有数据；

初值为 full(1)=0, full (2)=1, full (3)=0, full (4)=2；

empty(i) 表示 buffer(i) 是否有空间；

初值为 empty(1)=3, empty (2)=1, empty (3)=3, empty (4)=0 、

（3）进程程序

```
process  Proc(i)
  { ;
      p(full(i)) ;
      p(empty(i mod4 +1))
      p(m(i) && m(imod4)+1));
      move ;
      v(m(i) && m(i mod4)+1));
      v(full((i mod4 +1)));
      v(empty(i);
  }
```

例题

【解答题】（2021 统考真题）某银行提供 2 个服务窗口和 10 个供顾客等待的座位。顾客到达银行时，若有空座位，则到取号机上领取一个号，等待叫号。取号机每次仅允许一位顾客使用。当营业员空闲时，通过叫号选取一位顾客，并为其服务。

请添加必要的信号量和 PC 操作，实现上述过程。

例题

```
//当有两个服务窗口时
semaphore seats = 10;     //可容纳顾客数
semaphore requests = 2, service = 0;     //请求服务的相关信号量
semaphore mutex = 1;     //互斥取号
```

```
Customers (){
    while (1){
        P (seats);     //请求座位
        P (mutex);
        取 号;
        V (mutex);

        P (request);     //请求服务
        请求服务;
        V (service);     //通知营业员
    }
}
```

```
Employees (){
    while (1){
        P (service);     //初始为空，0则休息
        服 务
        V (seats)//释放座位
        服务完毕;
        V (request)//释放营业员，起身去接受服务
    }
}
```

例题

【解答题】理发店有 1 位理发师、 1 把理发椅和 n 把供等候理发的顾客坐的椅子。如果没有顾客，理发师便在理发椅上睡觉等第一个顾客到来。第一个顾客到来时，它必须叫醒理发师。如果理发师正在理发时又有顾客到来，则如果有空椅子可以坐，就坐下来等待，否则就离开。

例题

```
semaphore customres = 0;
semaphore barber= 0;     //初始💁睡觉
semaphore mutex = 1;
int chairs = n;     //初始容器（椅子）大小
int waiting = 0;     //初始可装入量（顾客量）
```

```
customers(){
    while(1){
        P(mutex);
        if (waiting < chairs){     //判断是否离开，若椅子满了转入else
            ++waiting;
            V(customers);    //         来客人了🤣唤醒理发师
            V(mutex);     //先解锁 以让顾客进来，
                          //来加大并发数。
            P(baber);     //请求理发
            等待理发;
        }else
            V(mutex);     //人满离开
    }
}
```

例题

```
semaphore customres = 0;
semaphore barber= 0;     //初始💁睡觉
semaphore mutex = 1;
int chairs = n;     //初始容器（椅子）大小
int waiting = 0;     //初始可装入量（顾客量）
```

```
baber(){      //💈
    while(1){
        P(customers);     //无人堵塞 睡觉，有人进行理发
        P(mutex);     //互斥变量
        --waiting;     //椅子释放
        V(baber);     //必须在互斥里，否则可能waiting一直减，
                      //导致baber只理发了一些人。

        理发;
    }
}
```

Chapter 6: Process Synchronization

- Background
- The Critical-Section Problem
- Peterson's Solution
- Synchronization Hardware
- Semaphores
- Classic Problems of Synchronization
- Monitors
- Synchronization Examples
- Atomic Transactions

Synchronization Examples

- Solaris
- Windows XP
- Linux
- Pthreads
- JAVA

Solaris Synchronization

- Implements a variety of locks to support multitasking, multithreading (including real-time threads), and multiprocessing （实现各种锁以支持多任务、多线程 (包括实时线程) 和多处理）
- Uses adaptive mutexes for efficiency when protecting data from short code segments. （在保护数据不受短代码段影响时，使用**自适应互斥信号量**提高效率）
- Uses condition variables and readers-writers locks when longer sections of code need access to data. （当较长的代码段需要访问数据时，**使用条件变量和读写锁**）
- Uses turnstiles （十字转门） to order the list of threads waiting to acquire either an adaptive mutex or a reader-writer lock.

Solaris Synchronization

- adaptive mutex is used to protect critical data
- On a multiprocessor （多处理器） system
  - spinlock （自旋锁）
    - If the lock is held by a thread currently running on another CPU, the thread spins while waiting for the lock to become available
  - Not spin （非自旋锁）
    - If the thread holding the lock is not running, the thread blocks until the releasing of the lock.
- On a single-processor （单处理器） system
  - The thread holding the lock is never running if the lock is tested by another thread, so threads always sleep rather than spin if they encounter a lock. （线程在遇到锁时总是休眠而不是旋转。）

Solaris Synchronization

- Reader-writer locks （读写锁） are used to protect data that are accessed frequently but are usually accessed in a read-only manner.
- More efficient than semaphore
- Used on only long sections of code.

Windows XP Synchronization

- For kernel thread （内核线程）
  - 单处理器： Uses interrupt masks （中断掩码） to protect access to global resources on uniprocessor systems
  - 多处理器： Uses spinlocks （自旋锁） on multiprocessor systems
- For user thread （用户线程）
  - provides dispatcher objects （调度对象） which may act as mutexes, semaphores and events
  - An event acts much like a condition variable （事件的作用很像条件变量）
  - An event may notify a waiting thread when a desired condition occurs. （事件发生可以通知等待的线程）

Linux Synchronization

- Prior to version 2.6, Linux was a nonpreemptive kernel （非抢占式内核） , meaning that a process running in kernel mode could not be peempted. （内核模式下运行的进程不能被抢占）
- Now the Linux kernel is fully preempted. （现在 Linux 内核被完全抢占了）
- Linux kernel provides:
  - Semaphores （信号量）
    - For single processor system （单处理器系统）
  - spin locks （自旋锁）
    - For SMP （对称多处理器系统）

Pthreads Synchronization

- Pthreads API is OS-independent （API 与操作系统无关）
- It provides:
  - mutex locks （互斥锁）
    - To protect critical section of code （保护临界区）
  - condition variables （条件变量）
    - behave much as described in monitor （与管程中的描述大致相同）
  - read-write locks （读写锁）

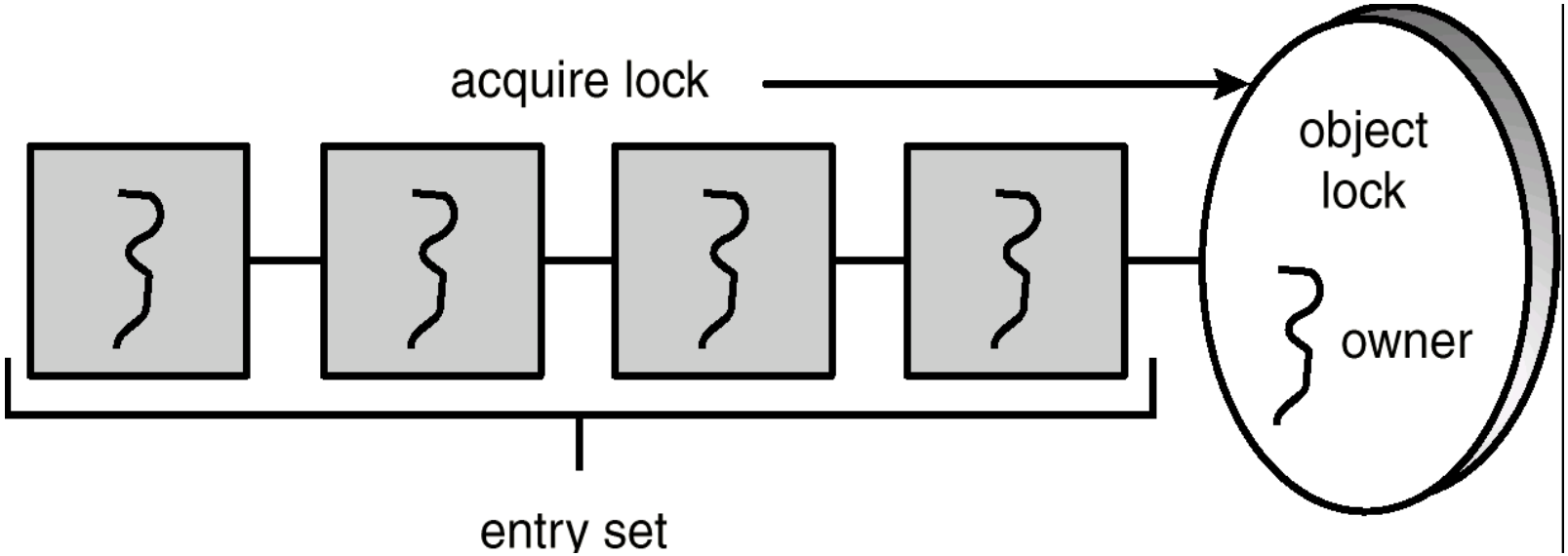
Java Synchronization

- Synchronized, wait(), notify() statements
- Multiple Notifications （多通知）
- Block Synchronization （块同步）
- Java Semaphores

Java Synchronization

- Synchronized statements （Synchronized 声明）
- Every object in JAVA has associated with it a single lock.
- When a method is declared as synchronized, calling the method requires owning the lock for the object. （当一个方法声明为“ synchronized”，那么调用这个方法就需要拥有该对象的锁）
- Entry set:
  - the set of threads waiting for the lock to become available. （如果对象的锁已被别的线程拥有，那么调用同步方法的线程进入该对象锁的入口等待队列 (entry set) 中）

Entry Set



synchronized enter() Method

```
public synchronized void enter(Object item) {
    while (count == BUFFER_SIZE)
            Thread.yield();
    ++count;
    buffer[in] = item;
    in = (in + 1) % BUFFER_SIZE;
}
```

synchronized remove() Method

```
public synchronized Object remove() {
    Object  item;
    while (count == 0)
            Thread.yield();
    --count;
    item = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
    return item;
}
```

Java Synchronization

- Producer-consumer problem （生产者消费者问题）：
  - The object BoundedBuffer has one lock （对象 BoundedBuffer 有一个锁）
  - A producer must acquire the lock before invoking method enter(). （生产者必须在调用方法之前获取锁）
  - When the lock is owned by a producer, if a consumer want to invoke remove(), it has to block until the producer exits the method enter() and release the lock. （当锁由生产者拥有时，如果消费者想要调用 remove()，它必须阻塞，直到生产者退出方法 enter() 并释放锁）

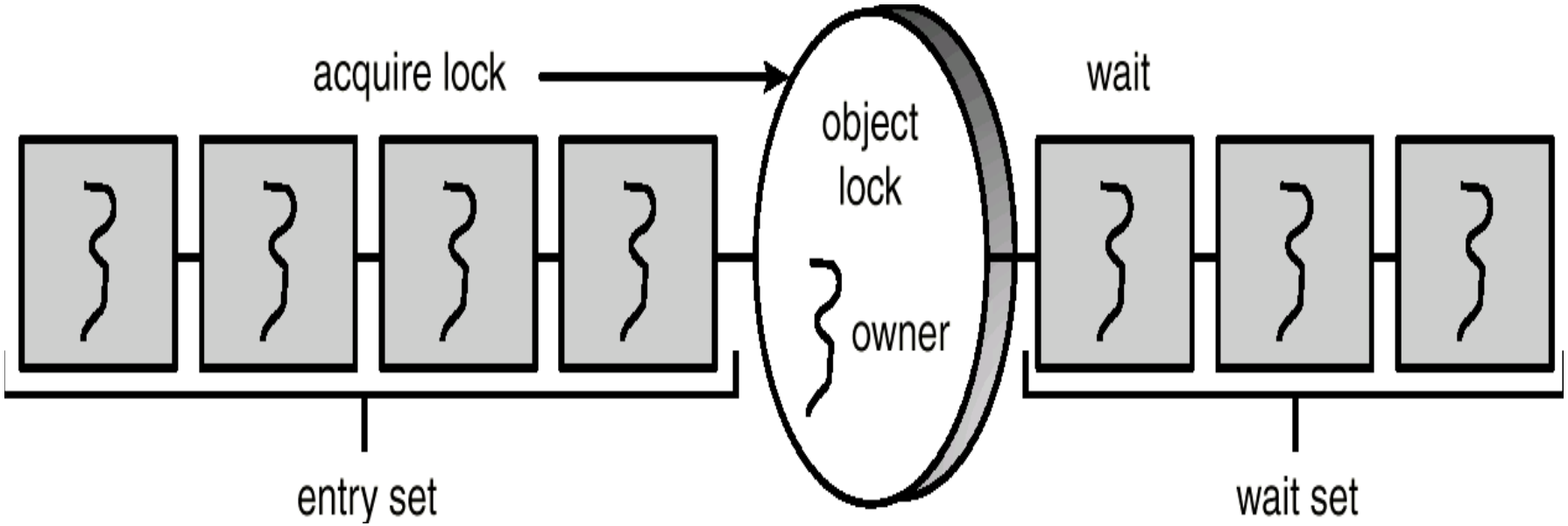
Java Synchronization

- However, lock ownership has led to another problem. （锁所有权带来了另一个问题）
- Assume that the buffer is full and the consumer is sleeping. （假设缓冲区已满，消费者处于睡眠状态）
- If the producer calls enter(), it will be allowed to continue because the lock is available. When the producer invokes enter(), it sees a full buffer and performs the yield(), but it still owns the lock for the object. （如果生产者调用 enter()，它将被允许继续，因为锁是可用的。当生产者调用 enter() 时，它看到一个满缓冲区并执行 yield()，但它仍然拥有对象的锁）
- When the consumer awakens and try to call remove(), it will block because it does not own the lock. （当消费者唤醒并尝试调用 remove() 时，它会阻塞，因为它不拥有锁）
- This leads to deadlock.

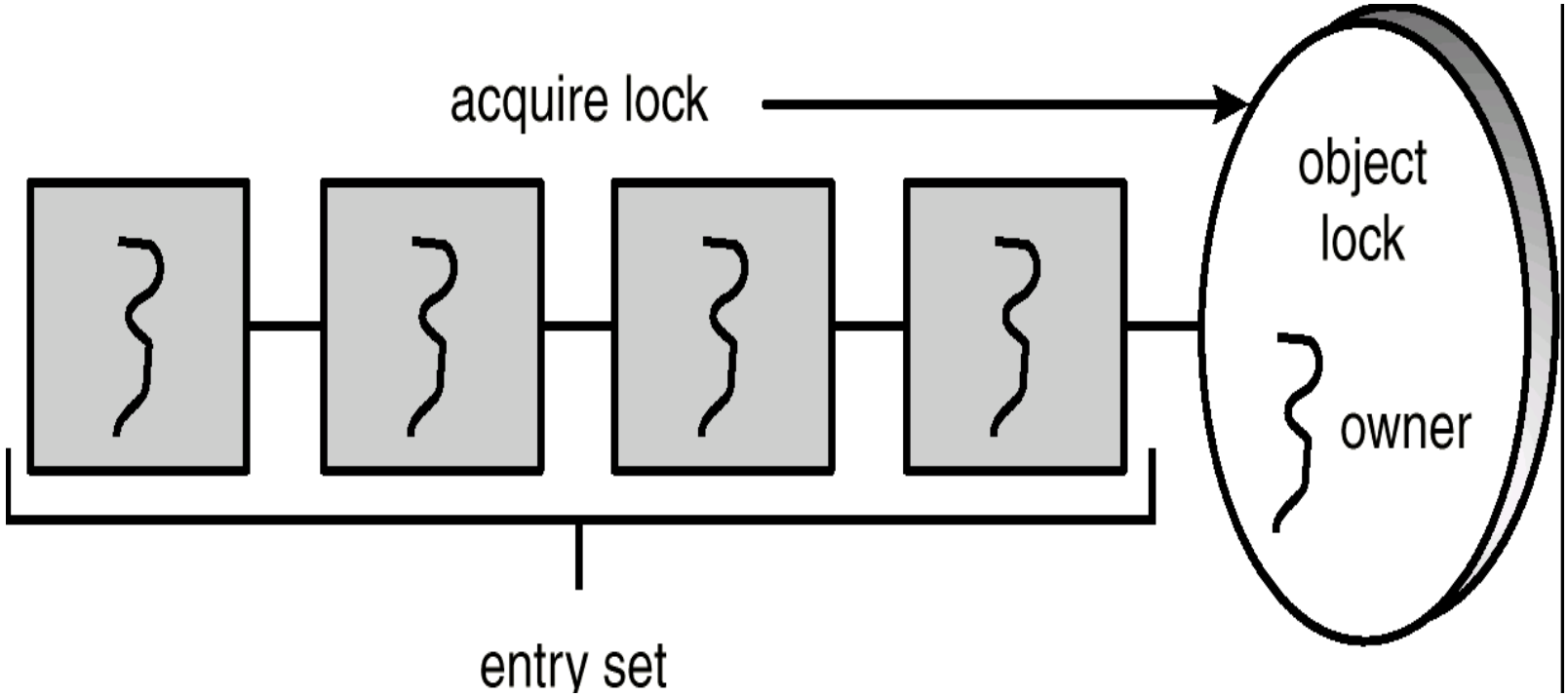
Java Synchronization

- Solution:
  - In addition to having a lock, every object also has a wait set.
  - Two new methods: wait() and notify().
- If the producer calls enter() and the buffer is full, it will release the lock by invoking wait(), and enter to the wait set for the object. （如果生产者调用 enter() 并且缓冲区已满，它将通过调用 wait() 释放锁，并进入对象的等待集）
- So the consumer can acquire the lock and call remove(), and then awake the producer by notify(). （消费者可以获取锁并调用 remove()，然后通过 notify() 唤醒生产者）

Entry and Wait Sets



Entry Set



The wait() Method

- When a thread calls wait(), the following occurs:
  - the thread releases the object lock. （线程释放锁）
  - thread state is set to blocked. （线程阻塞）
  - thread is placed in the wait set （放入等待集）

The notify() Method

- When a thread calls notify(), the following occurs:

  - selects an arbitrary thread T from the wait set （在等待集选择随便一个线程 T ）

  - moves T to the entry set. （将它挪入进入集）

  - sets T to Runnable. （将它设为就绪）

- T can now compete for the object's lock again （该线程能够继续竞争锁）

Multiple Notifications

- notify() selects an arbitrary thread T from the wait set
- If there are multiple threads in wait set and more than one condition for which to wait, it is possible to have another deadlock. （如果等待集中有多个线程，并且需要等待的条件不止一个，则可能再次发生死锁）
- 例：有 5 个线程 T1， T2， T3， T4， T5，一个共享变量 turn， 5 个线程都需要调用 dowork() 方法，规定只有编号与 turn 相同的线程才能进入 dowork() 。假定 turn=3， T1， T2， T4 在等待集中， T3 在调用 dowork() 。当 T3 完成了它的工作，它把 turn 设为 4，并调用 notify() 。 notify() 从等待集中任意选取一个线程，假设选中了 T2，但当 T2 恢复运行后检查条件时，发现 turn 并不是它的值，于是 T2 再次调用 wait(), 进入等待集。接下来，如果 T3， T5 调用 dowork()，也会进入等待集。这样， 5 个线程都进入了等待集，形成了死锁。

Multiple Notifications

- notify() selects an arbitrary thread from the wait set. This may not be the thread that you want to be selected. （可能没选择合适的 thread ）
- notifyAll() removes ALL threads from the wait set and places them in the entry set. This allows the threads to decide among themselves who should proceed next. （把所有等待集中的线程放入进入集）

The Readers-Writers Problem

- JAVA solution to reader-writer problem：
  - readerCount： number of readers
  - dbReading： is set to true if the database is currently being read
  - dbWriting： is set to true if the database is currently being written
  - startRead()、 endRead ()、 startWrite ()、 endWritet (): are all declared as synchronized to ensure mutual exclusion for the shared variables.

Reader Methods with Java Synchronization

```
public class Database {
   public Database() {
      readerCount = 0;
      dbReading = false;
      dbWriting = false;
   }
     public synchronized int startRead() { /* see next slides */ }
     public synchronized int endRead()  { /* see next slides */ }
     public synchronized void startWrite() { /* see next slides */ }
     public synchronized void endWrite()  { /* see next slides */ }

     private int readerCount;
     private boolean dbReading;
     private boolean dbWriting;
}
```

Writer Methods

```
public void synchronized startWrite() {
    while (dbReading == true || dbWriting == true)
            try {
                    wait();
            }
            catch (InterruptedException e) { }
            dbWriting = true;
}
public void synchronized endWrite() {
    dbWriting = false;
    notifyAll();
}
```

startRead() Method

```
public synchronized int startRead() {
    while (dbWriting == true) {
            try {
                    wait();
            }
            catch (InterruptedException e) { }
    }
    ++readerCount;
    if (readerCount == 1)
            dbReading = true;
    return readerCount;
}
```

endRead() Method

```
public synchronized int endRead() {
    --readerCount
    if (readerCount == 0)
      dbReading = false;
    notifyAll();
    return readerCount;
  }
```

Block Synchronization

- Blocks of code – rather than entire methods – may be declared as synchronized （可以将代码块——而不是整个方法——声明为同步的）
- This yields a lock scope that is typically smaller than a synchronized method （这产生的锁范围通常比同步方法小）

```
Object mutexLock = new Object();
   . . .
 public void someMethod() {
            // non-critical section
           synchronized(mutexLock) {
                  // critical  section
            }
           // non-critical section
   }
```

Java Semaphores

- Java does not provide a semaphore, but a basic semaphore can be constructed using Java synchronization mechanism （Java 不提供信号量，但是可以使用 Java 同步机制构造基本的信号量）

Semaphore Class

```
public class Semaphore {
   public Semaphore() {
            value = 0;
    }
   public Semaphore(int v) {
            value = v;
    }
   public synchronized void P() { /* see next slide */ }
   public synchronized void V() { /* see next slide */ }
   private int value;
}
```

P() Operation

```
public synchronized void P() {
    while (value <= 0) {
      try {
        wait();
      }
      catch (InterruptedException e) { }
    }
    value --;
  }
```

V() Operation

```
public synchronized void V() {
    ++value;
    notify();
}
```

Java Synchronization rules

- a thread that owns the lock for an object can enter another synchronized method for the same object. （拥有对象锁的线程可以进入同一对象另一个同步方法）
- a thread can nest synchronized method invocations for different objects. thus, a thread can simultaneously own the lock for several different objects （线程可以嵌套不同对象的同步方法调用。因此，一个线程可以同时拥有几个不同对象的锁）

Java Synchronization rules(Cont.)

- if a method is not declared as synchronized, then it can be invoked regardless of lock ownership, even while another synchronized method for the same object is executing. （如果一个方法没有被声明为 synchronized 的，那么不管锁的所有权如何，它都可以被调用，即使是在同一对象的另一个同步方法正在执行的时候。）
- if the wait set for an object is empty, then a call to notify() or notifyall() has no effect. （如果对象的等待集为空，则调用 notify （）或 notifyall （）不起作用）

例题

- 为什么在单处理器系统上通过禁止中断来实现同步原语的方法不适用于用户级程序?
- Explain why implementing synchronization primitives by disabling interrupts is not appropriate in a single-processor system if the synchronization primitives are to be used in user-level programs.

- 答:

如果用户程序禁止了中断，这个用户程序将永久霸占CPU，在单处理器系统上会使得整个系统崩溃

例题

- 进程的同步与互斥是由于程序的（ ）引起的：

A 顺序执行

B 长短不同

C 信号量

D 并发执行

答案： D

例题

- 自旋锁为什么不适用于单处理器系统，但在多处理器系统上经常用?
- Explain why spinlocks are not appropriate for single-processor systems yet are often used in multiprocessor systems.

- 答:

在单处理器上，线程的自旋会一直占用着唯一的 CPU 。

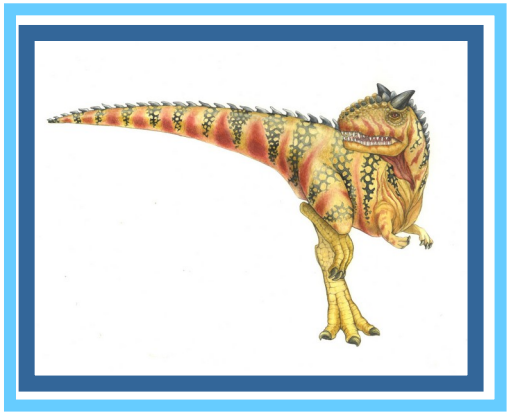
而多处理器有多个 CPU ，自旋不会阻塞锁的持有者

例题

- 与管程相关的 signal （）操作与为信号量定义的相应操作有何不同?
- How does the signal() operation associated with monitors differ from the corresponding operation defined for semaphores?

- 答：管程包括一个条件变量和其对应的等待队列
- 信号量的 signal() 是“累积的”，它会改变信号量的状态（增加计数值），这个状态变化是持久的，即使当时没有线程在等待。
- 管程的 signal() 是“非累积的”，它更像一个通知。如果发出通知时没有线程在条件变量上等待，那么这个通知就丢失了，不会产生任何效果。

End of Chapter 6



Silberschatz, Galvin and Gagne ©2009